

Learning Distributed Stabilizing Controllers for Multi-Agent Systems

Gangshan Jing[✉], He Bai[✉], *Member, IEEE*, Jemin George[✉], *Member, IEEE*, Aranya Chakraborty[✉], *Senior Member, IEEE*, and Piyush K. Sharma

Abstract—We address model-free distributed stabilization of heterogeneous continuous-time linear multi-agent systems using reinforcement learning (RL). Two algorithms are developed. The first algorithm solves a centralized linear quadratic regulator (LQR) problem without knowing any initial stabilizing gain in advance. The second algorithm builds upon the results of the first algorithm, and extends it to distributed stabilization of multi-agent systems with predefined interaction graphs. Rigorous proofs are provided to show that the proposed algorithms achieve guaranteed convergence if specific conditions hold. A simulation example is presented to demonstrate the theoretical results.

Index Terms—Reinforcement learning, linear quadratic regulator, optimal distributed control, multi-agent systems.

I. INTRODUCTION

REINFORCEMENT learning (RL) is a goal-oriented learning method where a system optimizes an intended policy according to a reward returned from its environment. Because of the generality of the approach, RL has found applications in diverse areas such as robotics [1], communication [2], electric power systems [3], and defense-related military applications [4]. It has also been shown as a fantastic tool for optimal control problems, especially for linear quadratic regulator (LQR) design with unknown system dynamics [5], [6]. A variety of formulations of RL has been

proposed in the model-free LQR literature such as adaptive dynamic programming (ADP) [7], Q-learning [8], [9], and zero-order optimization [10]. Extensions of these centralized designs to distributed RL have been reported in [11], [12], [13].

In this letter, we revisit the centralized LQR problem and the distributed stabilization of multi-agent networks with coupled dynamics using model-free RL. In the literature, almost all the RL algorithms for LQR design require a stabilizing controller to start the learning algorithm even when the plant dynamics is known. In practice, however, due to the uncertainty of dynamics, knowing such initial stabilizing gains may not always be possible. Accordingly, the novelty of our work is to design RL algorithms for generating centralized and distributed stabilizing controllers without knowing explicit system dynamics. The problem of learning centralized stabilizing controllers has been recently addressed in [9], [14] for discrete-time systems, but the same learning problem for continuous-time systems and distributed stabilization are yet to be studied.

To resolve this issue, we propose two off-policy RL algorithms based on the ADP technique for continuous-time linear systems with unknown dynamics. The first data-driven algorithm solves the centralized LQR problem without having any stabilizing gain as its initial guess. The second algorithm builds on the results of the first algorithm, and extends it to solving distributed stabilization of multi-agent systems with a predefined interaction graph. The fundamental idea is to introduce a damping parameter to the system, which is inspired by [15]. Our design, however, is quite different than [15] as we propose an *explicit* updating law for the damping parameter, followed by a rigorous proof of convergence. Moreover, unlike [15], in our work the distributed stabilizing gain is learned based on a centralized stabilizing gain. The distributed stabilizing controller learned by our algorithm can be utilized as the initial controller for distributed RL algorithms in the literature (e.g., [13]) to learn a suboptimal distributed controller. Simulation experiments are conducted to show the effectiveness of the proposed RL algorithms.

This rest of the paper is structured as follows. In Section II, we review the model-free LQR problem and the damping formulation, and introduce an off-policy RL algorithm without an initial stabilizing gain. In Section III, we extend this design to a model-free stabilization problem of multi-agent systems

Manuscript received March 4, 2021; revised April 1, 2021; accepted April 5, 2021. Date of publication April 8, 2021; date of current version June 25, 2021. This work was supported in part by the DEVCOM Army Research Laboratory Director's Research Award for External Collaborative Initiative under Grant Dec19-SE-021. The work of Gangshan Jing and Aranya Chakraborty was supported in part by the U.S. National Science Foundation (NSF) under Grant ECCS 1931932. Recommended by Senior Editor J.-F. Zhang. (*Corresponding author: Gangshan Jing.*)

Gangshan Jing and Aranya Chakraborty are with the Department of Electrical and Computer Engineering, North Carolina State University, Raleigh, NC 27695 USA (e-mail: gjing@ncsu.edu; achakra2@ncsu.edu).

He Bai is with the Department of Mechanical and Aerospace Engineering, Oklahoma State University, Stillwater, OK 74078 USA (e-mail: he.bai@okstate.edu).

Jemin George and Piyush K. Sharma are with the Computational & Information Sciences Directorate, DEVCOM Army Research Laboratory, Adelphi, MD 20783 USA (e-mail: jemin.george.civ@mail.mil; piyush.k.sharma.civ@mail.mil).

Digital Object Identifier 10.1109/LCSYS.2021.3072007

2475-1456 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

with coupled dynamics. In Section IV, a simulation example is presented to illustrate the effectiveness of the two algorithms. In Section V, conclusions are drawn. The proof of Theorem 1 is presented in the Appendix.

Notation: Throughout the paper, given a matrix X , $X \succeq 0$ implies that X is positive semi-definite; $X \succ 0$ implies that X is positive definite; $\mathbb{S}_+$ is the set of positive definite matrices. For a symmetric matrix X , $\lambda_{\max}(X)$ and $\lambda_{\min}(X)$ denote the maximum and minimum eigenvalues of X , respectively. Let $\text{vec}(X) \in \mathbb{R}^{n^2}$ denote the vector stacking up columns of $X \in \mathbb{R}^{n \times n}$ from left to right. Let $\text{diag}[X_1, \dots, X_N]$ denote a block diagonal matrix with X_i 's on the diagonal. $\mathbb{N}_+$ denotes the set of positive integers. The Euclidean 2-norm is denoted by $\|\cdot\|$.

II. MODEL-FREE LQR WITHOUT AN INITIAL STABILIZING CONTROLLER

Consider the continuous-time LQR problem:

$$\begin{aligned} \min_K J(x(0), K) &= \int_0^\infty (x^T Q x + u^T R u) dt \\ \text{s.t. } \dot{x}(t) &= A x(t) + B u(t), \\ u(t) &= -K x(t), \end{aligned} \quad (1)$$

where $x(t) \in \mathbb{R}^n$ and $u(t) \in \mathbb{R}^m$ are the system state and the control input at time t , respectively, $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times m}$ are unknown. The variable K is the control gain matrix, which is also referred to as the *policy*. $Q \succeq 0$ and $R \succ 0$ are both considered to be known. Such a setting is reasonable because Q and R determine the desired transient behavior of the system, thus are usually artificially designed.

We make the following assumption to guarantee the existence and uniqueness of the solution to (1):

Assumption 1: (A, B) is stabilizable, $(Q^{1/2}, A)$ is observable.

The problem we aim to solve is formally stated as follows:

Problem 1: Given Q, R , and a collection of input and state data $\mathcal{D} = \{x(t), u(t), t \in [0, Z\delta t]\}$ with $\delta t > 0$ and $Z \in \mathbb{N}_+$ for the linear continuous-time system in (1), find the optimal control gain K for the LQR problem (1).

A. The Damped LQR Problem and Kleinman's Algorithm

In this subsection, we review the damped LQR problem in [15] and Kleinman's Algorithm in [16].

The damped LQR problem is as follows:

$$\begin{aligned} \min_K J(\hat{x}(0), K, \alpha) &= \int_0^\infty e^{-2\alpha t} (\hat{x}^T Q \hat{x} + \hat{u}^T R \hat{u}) dt \\ \text{s.t. } \dot{\hat{x}}(t) &= A \hat{x}(t) + B \hat{u}(t), \\ \hat{u}(t) &= -K \hat{x}(t), \end{aligned} \quad (2)$$

where $\alpha \geq 0$ is the damping parameter. By setting $x(t) = e^{-\alpha t} \hat{x}(t)$ and $u(t) = e^{-\alpha t} \hat{u}(t)$, problem (2) is equivalent to the following problem in the sense that with the same control policy and $x(0) = \hat{x}(0)$, they have equivalent cost function values:

$$\begin{aligned} \min_K J(x(0), K, \alpha) &= \int_0^\infty (x^T Q x + u^T R u) dt \\ \text{s.t. } \dot{x}(t) &= (A - \alpha I_n) x(t) + B u(t), \end{aligned}$$

$$u(t) = -K x(t). \quad (3)$$

When A and B are known, given α such that $(A - \alpha I_n, B)$ is stabilizable, it is well-known that the solution to (3) is

$$K^* = R^{-1} B^T P, \quad (4)$$

where P is the solution to the following algebraic Riccati equation:

$$(A - \alpha I_n)^T P + P(A - \alpha I_n) + Q - P B R^{-1} B^T P = 0. \quad (5)$$

Given a stabilizing gain K_0 , Kleinman's algorithm [16] shows that equation (5) can be obtained by alternatively solving for P_k and K_k such that

$$(A - \alpha I_n - B K_k)^T P_k + P_k (A - \alpha I_n - B K_k) + Q + K_k^T R K_k = 0, \quad (6)$$

$$K_{k+1} = R^{-1} B^T P_k, \quad k = 0, 1, 2, \dots \quad (7)$$

The updating mechanism (6)-(7) can be viewed as *policy iteration*, implying that Kleinman's algorithm is a model-based RL algorithm. Let

$$V_k(x(0)) = x^T(0) P_k x(0)$$

be the cost to go function evaluating the cost in (3) with $u(t) = K_{k+1} x(t)$ as the controller. A good property of the iteration algorithm is the monotonic decreasing of P_k during the evolution:

$$P_k \preceq P_{k-1}. \quad (8)$$

As a result, K_{k+1} is still stabilizing and actually yields a better performance. Solving for P_k from (6) is *policy evaluation*, and the updating of K_{k+1} in (7) is *policy improvement*.

B. Model-Free RL for the Damped LQR

In this subsection, by applying the model-free RL algorithm in [7] to the damped LQR problem, we achieve policy iteration without having the explicit information of the dynamics model, while instead, based on control input and state data only. It is indeed feasible to first estimate A and B by data set $\mathcal{D}$, then find the optimal controller by implementing Kleinman's algorithm. However, such a framework may cause extra computational errors and a longer learning time [6]. In our work, we will focus on how to learn the optimal or stabilizing controller directly from the control input and state data $\mathcal{D}$.

Let $A_k = A - B K_k$. Based on system dynamics in (3), the updating laws of P_k and K_{k+1} , we study the value function change for a given policy K_{k+1} during the evolution of the original system in (1) in time interval $[t, t + \delta t]$:

$$\begin{aligned} & x^T(t + \delta t) P_k x(t + \delta t) - x^T(t) P_k x(t) \\ &= \int_t^{t+\delta t} [x^T (A_k^T P_k + P_k A_k) x + 2(u + K_k x)^T B^T P_k x] ds \\ &= \int_t^{t+\delta t} x^T (2\alpha P_k - Q - K_k^T R K_k) x ds \\ &\quad + 2 \int_t^{t+\delta t} (u + K_k x)^T R K_{k+1} x ds, \end{aligned}$$

where the last equality is obtained based on (6) and (7).

Define the mapping $\mu(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^{n(n+1)/2}$ on vector $y = (y_1, \dots, y_n)^T \in \mathbb{R}^n$ and the mapping $\nu(\cdot) : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}^{n(n+1)/2}$ on matrix $X = [X_{ij}]$ such that

$$\begin{aligned}\mu(y) &= (y_1^2, y_1 y_2, \dots, y_1 y_n, y_2^2, y_2 y_3, \dots, y_{n-1} y_n, y_n^2)^T, \\ \nu(X) &= (X_{11}, 2X_{12}, \dots, 2X_{1n}, X_{22}, 2X_{23}, \dots, X_{n-1,n}, X_{nn})^T.\end{aligned}$$

We also let

$$\begin{aligned}\delta_{xx} &= [\mu(x)|_0^{\delta t}, \dots, \mu(x)|_{(Z-1)\delta t}^{Z\delta t}]^T \in \mathbb{R}^{Z \times n(n+1)/2}, \\ I_x &= \left[\int_0^{\delta t} \mu(x) ds, \dots, \int_{(Z-1)\delta t}^{Z\delta t} \mu(x) ds \right]^T \in \mathbb{R}^{Z \times n(n+1)/2}, \\ I_{xx} &= \left[\int_0^{\delta t} (x \otimes x) ds, \dots, \int_{(Z-1)\delta t}^{Z\delta t} (x \otimes x) ds \right]^T \in \mathbb{R}^{Z \times n^2}, \\ I_{xu} &= \left[\int_0^{\delta t} (x \otimes u) ds, \dots, \int_{(Z-1)\delta t}^{Z\delta t} (x \otimes u) ds \right]^T \in \mathbb{R}^{Z \times mn}.\end{aligned}$$

Then equation (9) is equivalent to

$$\Theta_k(\alpha) \begin{pmatrix} \nu(P_k) \\ \text{vec}(K_{k+1}) \end{pmatrix} = \Xi_k, \quad (9)$$

where $\Theta_k(\alpha) = (\delta_{xx} - 2\alpha I_x, -2I_{xu}(I_n \otimes R) - 2I_{xx}(I_n \otimes K_k^T R)) \in \mathbb{R}^{Z \times [n(n+1)/2 + mn]}$, $\Xi_k = -I_x \nu(Q + K_k^T R K_k) \in \mathbb{R}^Z$.

Assumption 2: $\text{Rank}((I_x, I_{xu})) = \frac{n(n+1)}{2} + mn$.

As shown in [7], if the data set $\mathcal{D}$ is accurate and satisfies Assumption 2, then (9) can be exactly solved, and the solution corresponds to P_k and K_{k+1} obtained according to (6) and (7). However, in numerical experiments, usually $\mathcal{D}$ is inaccurate. Therefore, we solve (9) by treating it as a least squares problem. Moreover, in numerical simulations, sometimes Assumption 2 holds but the smallest eigenvalue of $M^T M$ is close to zero ($M = (I_x, I_{xu})$), which may induce undesired solutions to the least squares problem. To overcome this issue, we need to collect enough amount of control input and state data such that $M^T M$ is well-conditioned. Note that the data set $\mathcal{D}$ can be independent of the control policy K_k to be updated. Hence, the RL algorithm based on solving (9) is *off-policy*.

C. RL Without Stabilizing Initialization

It has been shown in [7] that under Assumptions 1 and 2, with a stabilizing gain at hand, the optimal controller K^* for problem (1) can be obtained by repeatedly solving (9) with $\alpha = 0$. In this subsection, we propose a model-free approach for seeking K^* without having a stabilizing gain in advance.

Note that for a sufficiently large α , $A - \alpha I_n$ is always stable and thus $(A - \alpha I_n, B)$ is stabilizable. In this work, we will start with a sufficiently large α_0 such that $K(0) = \mathbf{0}_{m \times n}$ is stabilizing, i.e., $A - \alpha_0 I_n$ is Hurwitz. Then we decrease α_k until $(A - \alpha_k I_n, B)$ is not stabilizable. Under Assumption 1, α_k converges to 0 as $k \rightarrow \infty$. Different from [15], we will propose a model-free approach and specify the updating laws for α and K .

The fundamental idea is to update α_{k+1} and K_{k+1} alternately. During each iteration, we firstly find the minimum α_{k+1} such that $A - BK_k - \alpha_{k+1} I_n$ is stable, then update K_{k+1} such that $A - BK_{k+1} - \alpha_{k+1} I_n$ is stable. The minimum α_{k+1} can be

Algorithm 1 Alternating RL Algorithm for Problem 1

Input: $Q, R, \mathcal{D} = \{x(t), u(t), t \in [0, Z\delta t]\}$, $\eta = \alpha_0/S$ with $S \in \mathbb{N}_+$, $\alpha_0 > 0$, threshold $\epsilon > 0$.

Output: K^* .

1. Set $k \leftarrow 0$, $K_0 \leftarrow \mathbf{0}_{m \times n}$, and $P_{-1} \leftarrow \mathbf{0}_{n \times n}$. Compute δ_{xx} , I_{xx} , I_{xu} and Ξ_k .
2. **while** $\alpha_k \geq \eta$
Obtain α_{k+1} , P_k and K_{k+1} by implementing Algorithm 2, set $k \leftarrow k+1$.
end while
3. **while** $\frac{\|K_{k+1} - K_k\|}{\|K_k\|} \geq \epsilon$
Set $\alpha_{k+1} = 0$. Obtain P_k and K_{k+1} by solving (9).
Set $k \leftarrow k+1$.
end while
4. Set $K^* = K_{k+1}$.

Algorithm 2 Updating α_{k+1} , P_k , K_{k+1} for Problem 1

Input: δ_{xx} , I_{xx} , I_{xu} , Ξ_k , $P = P_{k-1}$, $K = K_k$, α_k , $\sigma > 0$, step size $\eta = \alpha_0/S$ with $S \in \mathbb{N}_+$.

Output: α_{k+1} , P_k , and K_{k+1} .

1. Set $l = 0$.
2. **while** $\alpha_k \geq \eta$
3. Set $\alpha_{k+1} \leftarrow \alpha_k - \eta$. Compute $\Theta_k(\alpha_{k+1})$.
4. Obtain P_k and K_{k+1} by solving (9).
5. **if** $P_k > 0$ and $\|P_k - P_{k-1}\| < \sigma$
Set $\alpha_k \leftarrow \alpha_{k+1}$, $P \leftarrow P_k$, $K \leftarrow K_{k+1}$, $l \leftarrow l+1$
else
Go to step 7.
end if
6. **end while**
7. **if** $l = 0$
Set $\alpha_{k+1} \leftarrow \alpha_k$, obtain P_k and K_{k+1} by solving (9).
else
Set $\alpha_{k+1} \leftarrow \alpha_k$, $P_k \leftarrow P$, $K_{k+1} \leftarrow K$.
end if

obtained by solving the following optimization:

$$\begin{aligned}\min_{\alpha_{k+1}, P_k, K_{k+1}} \quad & \alpha_{k+1} \\ \text{s.t.} \quad & \alpha_{k+1}, P_k, K_{k+1} \text{ satisfies (9),} \\ & \|P_k - P_{k-1}\| < \sigma, \\ & P_k > 0, \alpha_{k+1} \geq 0,\end{aligned} \quad (10)$$

where $\sigma > 0$ is a predefined constant. Here σ can be any positive constant. Note that if σ is very large, the solution α_{k+1} to (10) may render $A - \alpha_{k+1} I_n - BK_k$ very close to an unstable matrix, thereby leading to a matrix P_k with very large eigenvalues. Hence, the second constraint in (10) is to restrict $\|P_k\|$.

The solution to (10) always makes $A - \alpha_{k+1} I_n - BK_k$ stable because the corresponding P_k , which is actually the solution to (6) for α_{k+1} , is positive definite. Moreover, the corresponding K_{k+1} must be stabilizing for $A - \alpha_{k+1} I_n$ because it is actually the policy improvement result (7).

Observe that both α_{k+1} and P_k are variables in (10), making the optimization (10) nonlinear. In fact, in our algorithm, we do not need the exactly optimal solution to (10). Hence, we will solve for an approximate optimal solution by using Algorithm 2. The RL algorithm for solving Problem 1 is given in Algorithm 1, which contains Algorithm 2 as the policy improvement step.

Remark 1: Since $\eta = \alpha_0/S$ in Algorithm 2, theoretically α_k converges to 0 if it is decreased by η for S times. In numerical simulations, there may be errors when computing η . As a result, α_k may be a small scalar when $\alpha_k < \eta$. However,

this error can be avoided by first giving η , and then selecting a large enough S such that $\alpha_0 = S\eta$ renders $A - \alpha_0 I_n$ stable. Then for any step k , $\alpha_k = 0$ if $\alpha_k < \eta$.

Theorem 1: Suppose $Q \succ 0$. Under Assumptions 1, 2, by implementing Algorithm 1, there exist $\eta > 0$ and $S_0 > 0$, such that if $S \geq S_0$, then K_k converges to the optimal control gain as $k \rightarrow \infty$.

Remark 2: In practice, matrix Q in the LQR problem may not satisfy the positive definiteness assumption. In this scenario, we can first obtain a stabilizing gain by implementing Algorithm 1 with an artificially designed positive definite Q' , then run a conventional ADP algorithm, e.g., [7], to learn the optimal controller for the original LQR problem.

III. LEARNING A DISTRIBUTED STABILIZING CONTROLLER

Suppose that the system in (1) is a heterogeneous multi-agent system with N agents, where the system equation for each agent is written as

$$\dot{x}_i = A_{ii}x_i + \sum_{j \in \mathcal{N}_i} A_{ij}x_j + B_i u_i, \quad i = 1, \dots, N, \quad (11)$$

where $x_i \in \mathbb{R}^{\bar{n}_i}$ and $u_i \in \mathbb{R}^{\bar{m}_i}$ are the state and control input of agent i , respectively, $A_{ij} \in \mathbb{R}^{\bar{n}_i \times \bar{n}_j}$, and $\mathcal{N}_i$ is the set of agents whose dynamics are coupled with agent i . Note that different agents may have different dimensions for states and control inputs. Moreover, the formulation (11) contains a special case where different agents have decoupled dynamics.

Remark 3: In (11), the dynamics of each agent includes neighbors' states (which is for achieving coordination such as consensus and formation control [11]) and an additive input that provides extra freedom to this agent [17]. Moreover, the coupled dynamics can describe internal dynamics of a multi-agent system, e.g., inter-connected power systems [18].

Define $n = \sum_{i=1}^N \bar{n}_i$ and $m = \sum_{i=1}^N \bar{m}_i$. Let $x = (x_1^T, \dots, x_N^T)^T$, $u = (u_1^T, \dots, u_N^T)^T$, $A = [A_{ij}] \in \mathbb{R}^{n \times n}$ with $A_{ij} = \mathbf{0}_{\bar{n}_i \times \bar{n}_j}$ for $(i, j) \notin \mathcal{E}$ and $B = \text{diag}\{B_1, \dots, B_N\} \in \mathbb{R}^{n \times m}$. Then we are able to write the compact system dynamics as

$$\dot{x} = Ax + Bu. \quad (12)$$

Let an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote the communication graph interpreting the desired interaction relationship among agents. Here $\mathcal{V} = \{1, \dots, N\}$ is the set of agents, $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ is the set of edges specifying those pairs of agents that are able to utilize information of each other. Our goal is to find a structured stabilizing gain matrix $K \in \mathbb{R}^{m \times n}$ for the multi-agent system (11) such that the controller of agent i determined by $u = -Kx$ involves the state of agent j if and only if $(i, j) \in \mathcal{E}$. In other words, the goal is to find K such that $A - BK$ is stable, and $K \in \mathcal{S}_K(\mathcal{G})$, where the set $\mathcal{S}_K(\mathcal{G})$ is defined as follows:

$$\mathcal{S}_K(\mathcal{G}) = \{K \in \mathbb{R}^{m \times n} : K(i, j) = \mathbf{0}_{\bar{m}_i \times \bar{n}_j} \text{ if } (i, j) \notin \mathcal{E}\}, \quad (13)$$

where $K(i, j)$ is a submatrix of K composed of elements from the $(\sum_{k=1}^{i-1} \bar{m}_k + 1)$ -th to $\sum_{k=1}^i \bar{m}_k$ -th rows and from $(\sum_{l=1}^{j-1} \bar{n}_l + 1)$ -th to $\sum_{l=1}^j \bar{n}_l$ columns of K .

The problem to be solved is formally stated as follows.

Problem 2: Given a collection of data $\mathcal{D} = \{x(t), u(t), t \in [0, Z\delta t]\}$ with $\delta t > 0$ and $Z \in \mathbb{N}_+$ for the linear continuous-time system in (11), find a control gain matrix $K_d \in \mathcal{S}_K(\mathcal{G})$ such that $A - BK_d$ is stable.

Under Assumptions 1 and 2, by artificially designing a cost function in (1) with $Q = I_n$ and $R = I_m$, and implementing Algorithm 1, we can obtain a stabilizing gain K_s when α_k converges to 0. Next we show how to obtain a distributed stabilizing gain K_d based on a stabilizing gain K_s .

Similar to (13), define

$$\begin{aligned} \mathcal{S}_P(\mathcal{G}) &= \{P \in \mathbb{R}^{n \times n} : P(i, j) = \mathbf{0}_{\bar{n}_i \times \bar{n}_j} \text{ if } (i, j) \in \mathcal{E}, \\ \mathcal{S}_R &= \{R \in \mathbb{R}^{m \times m} : R(i, j) = \mathbf{0}_{\bar{m}_i \times \bar{m}_j} \text{ if } i \neq j\}. \end{aligned}$$

Lemma 1: There exists a distributed stabilizing gain $K_d \in \mathcal{S}_K$ for (11) if the following problem is feasible:

$$\begin{aligned} &\text{find } P \\ &\text{s.t. } (A - BK_s)^T P + P(A - BK_s) < 0, \\ &P \in \mathcal{S}_P(\mathcal{G}), \quad P \succ 0. \end{aligned} \quad (14)$$

Proof: Suppose that P_d is a solution to (14). Denote

$$D = -(A - BK_s)^T P_d + P_d(A - BK_s) \succ 0.$$

For any $R' \succ 0$ such that $R' \in \mathcal{S}_R$, choose $s > 0$ such that $sD \succeq K_s^T R' K_s$. Then sP_d is still a solution to (14). Let $Q' = sD - K_s^T R' K_s$. Let $K_d = sR'^{-1} B^T P_d$. We can view K_d as a policy improvement from the current policy K_s for the following cost function:

$$J_k(x(0), u) = \int_0^\infty (x^T Q' x + u^T R' u) dt. \quad (15)$$

Therefore, K_d is stabilizing as well. Since B and R' are block-diagonal and $P \in \mathcal{S}_P(\mathcal{G})$, we have $K_d \in \mathcal{S}_K$. ■

In a model-free way, similar to the last section, (14) can be transformed to the data-based form:

$$\begin{aligned} &\text{find } P_d, D, E \\ &\text{s.t. } \Theta(K_s) \begin{pmatrix} v(P_d) \\ \text{vec}(E) \end{pmatrix} = -I_x v(D), \\ &D \succ 0, P_d \in \mathcal{S}_P, P_d \succ 0, \end{aligned} \quad (16)$$

where $\Theta(K_s) = (\delta_{xx}, -2I_{xu}(I_n \otimes R) - 2I_{xx}(I_n \otimes K_s^T R)) \in \mathbb{R}^{Z \times [n(n+1)/2 + mn]}$. According to (9), we know that for any solution $\{P_d, D, E\}$ to (16), it actually holds that $E = R^{-1} B^T P_d$. Here matrix $R \succ 0$ is artificially designed and will be used to compute $\Theta(K_s)$.

Next we transform (16) to a semi-definite program (SDP), by converting positive definite constraints to positive semi-definite constraints. Note that for any solution $\{P_d, D, E\}$ to (16), $\{sP_d, sD, sE\}$ for any $s > 0$ is still a solution. That is, once (14) is feasible, there must exist a solution $\{P_d, D, E\}$ to (14) such that $D \succeq cI_n$ and $P_d \succeq cI_n$ for any $c > 0$. In practice, there may be a desired range for the traces of the solution matrices. Without loss of generality, we aim to solve the following linear SDP:

$$\begin{aligned} &\min_{P_d, D, E} \text{trace}(P_d) \\ &\text{s.t. } \Theta(K_s) \begin{pmatrix} v(P_d) \\ \text{vec}(E) \end{pmatrix} = -I_x v(D), \end{aligned}$$

$$D \geq cI_n, P_d \in \mathcal{S}_P(\mathcal{G}), P_d \geq cI_n, \quad (17)$$

where $c > 0$ is artificially given depending on the requirement in practice.

Lemma 2: Under Assumption 2, (14) is feasible if and only if (17) is feasible.

Proof: Sufficiency. By following similar lines to the proof of [7, Lemma 6], the validity of Assumption 2 implies that a solution $\{P_d, D, E\}$ to (17) must satisfy

$$-D = (A - BK_s)^T P_d + P_d(A - BK_s). \quad (18)$$

Therefore, P_d must be a solution to (14).

The necessity can be proved by noting that for any solution P_d to (14), sP_d with any $s > 0$ is still a solution. ■

Now we present Algorithm 3 as the algorithm for learning a distributed stabilizing controller for (11).

The block-diagonal input matrix R' in Algorithm 3 can be designed depending on the requirement in practice, which affects the obtained control gain. The simplest way to design R' is setting $R' = I_m$. The coefficient s in Algorithm 3 makes the resulting K_d stabilizing as long as it satisfies (19).

Theorem 2: Under Assumption 2, if (14) is feasible, then Algorithm 3 solves Problem 2.

Proof: The formula for s in (19) ensures $sD \geq K_s^T R' K_s$. By following similar lines to the proof of [7, Lemma 6], a solution $\{P_d, D, E\}$ to (17) satisfies $E = R^{-1} B^T P_d = B^T P_d$. Together with the proof of Lemma 1, we obtain that $K_d = sR'^{-1} E = sR'^{-1} B^T P_d \in \mathcal{S}_K$ is stabilizing. ■

Remark 4: Algorithms 1 and 3 are both centralized because both (9) and (17) involve the overall control input u and state x for the system. However, the controller obtained by implementing Algorithm 3 is distributed, because the controller of each agent only involves its neighbors' states.

IV. SIMULATIONS

Consider a multi-agent system (11) with 3 agents. The overall system matrix is given as

$$A = \begin{pmatrix} 0.48 & 0.23 & 0.89 & 0.86 & 0 & 0 \\ 0.12 & 0.07 & 0.16 & 0.73 & 0 & 0 \\ 0.64 & 0.03 & 0.57 & 0.71 & 0.65 & 0.30 \\ 0.47 & 0.16 & 0.62 & 0.25 & 0.13 & 0.67 \\ 0 & 0 & 0.40 & 0.95 & 0.11 & 0.63 \\ 0 & 0 & 0.14 & 0.69 & 0.90 & 0.08 \end{pmatrix},$$

and the control input matrix is

$$B = \text{blkdiag} \left\{ \begin{pmatrix} 0.37 \\ 0.92 \end{pmatrix}, \begin{pmatrix} 0.09 \\ 0.52 \end{pmatrix}, \begin{pmatrix} 0.91 \\ 0.31 \end{pmatrix} \right\} \in \mathbb{R}^{6 \times 3}.$$

The interaction topology is considered to be consistent with inter-agent dynamics coupling relationship reflected by matrix A . By setting $\alpha_0 = 2.46$, $\sigma = 100$, $\eta = 0.001$, $Q = I_6$, $R = I_3$, the result of implementing Algorithm 1 is shown in Fig. 1. Observe that as α_{k+1} evolves, $\lambda_{\max}(P_k)$ may be increased or decreased because every time α_{k+1} decreases, the eigenvalues of $A - \alpha I_n - BK_k$ are closer to 0. Once α_{k+1} converges to 0, $\lambda_{\max}(P_k)$ keeps non-increasing, and ultimately converges to $\lambda_{\max}(P^*)$, where P^* is the optimal cost to go matrix for (1).

Algorithm 3 RL Algorithm for Problem 2

Input: $\mathcal{D} = \{x(t), u(t), t \in [0, Z\delta t]\}$, $R' \in \mathbb{S}_+ \cap \mathcal{S}_R$, $\alpha_0 > 0$, $c > 0$.

Output: K_d .

1. Set $Q = I_n$, $R = I_m$. Implement Algorithm 1 until $\alpha_k = 0$, obtain a stabilizing gain K_s .
2. Solve SDP (17), obtain solution $\{P_d, D, E\}$. Let $K_d = sR'^{-1}E$, where

$$s \geq \frac{\lambda_{\max}(K_s^T R' K_s)}{\lambda_{\min}(D)}. \quad (19)$$

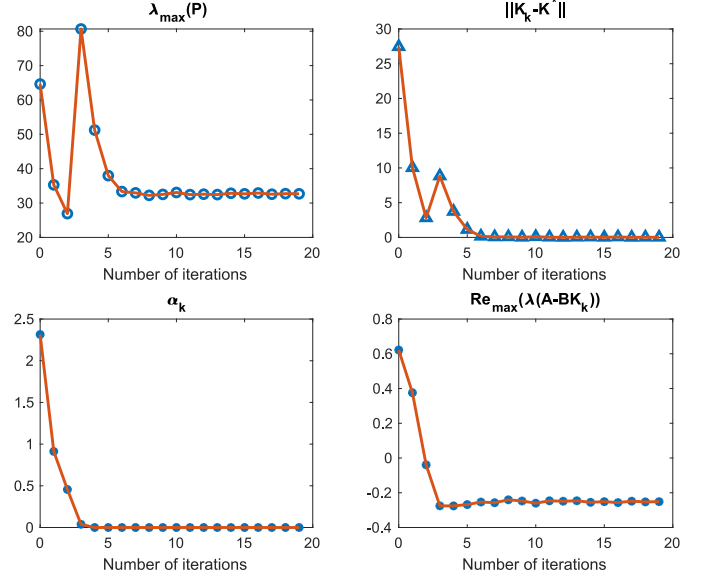


Fig. 1. The results by implementing Algorithm 1.

Moreover, K_k asymptotically converges to the optimal control gain, which is as follows:

$$K^* = \begin{pmatrix} 3.51 & 0.86 & 3.82 & 2.53 & 0.62 & 0.23 \\ 4.36 & 0.05 & 5.59 & 4.34 & 1.63 & 1.32 \\ 1.75 & -0.01 & 3.17 & 3.09 & 2.45 & 2.18 \end{pmatrix}.$$

By taking K^* as the stabilizing gain matrix K_s and implementing Algorithm 3 with $c = 100$ and $R' = I_3$, the obtained distributed stabilizing control gain K_d is

$$\begin{pmatrix} 139.55 & 102.25 & 73.54 & 33.76 & 0 & 0 \\ 174.73 & -44 & 165.15 & 142.76 & 4.52 & 3.97 \\ 0 & 0 & 91.70 & -5.61 & 179.35 & 91.79 \end{pmatrix}.$$

The structure of K_d shows that the controllers of agents 1 and 3 do not involve state information of each other.

V. CONCLUSION

We have proposed two off-policy model-free RL algorithms for the optimal control of general linear systems and for the distributed stabilization of linear multi-agent systems. By introducing a damping parameter, the RL-based LQR control methods can be generalized to scenarios where an initial stabilizing gain is not available. For a multi-agent system, a distributed stabilizing gain with a desired distributed structure can be learned based on a centralized stabilizing gain. Future work includes developing distributed learning algorithms, considering coupled inputs in the dynamics, and extension to directed interaction graphs.

APPENDIX PROOF OF THEOREM 1

Before entering into the proof, we first present a supporting lemma. Let $K^*(\alpha)$ be the optimal control gain for (2). The following lemma shows a robustness property of $K^*(\alpha)$.

Lemma 3: Suppose that $Q > 0$. Given $\alpha_0 > 0$, there exists $\delta_1 > 0$, such that for any $\alpha \in [0, \alpha_0]$, if $\|K - K^*(\alpha)\| < \delta_1$, then $A - \alpha I - BK$ is stable.

Proof: For simplicity, we denote $A_\alpha = A - \alpha I_n$, $K_\alpha^* = K^*(\alpha)$. From the definition of $K^*(\alpha)$, there exists $P_\alpha > 0$, such that

$$(A_\alpha - BK_\alpha^*)^T P_\alpha + P_\alpha (A_\alpha - BK_\alpha^*) + Q + K_\alpha^{*T} R K_\alpha^* = 0, \quad (20)$$

Let $\Delta K = K^* - K$, we have

$$\begin{aligned} & (A_\alpha - BK)^T P_\alpha + P_\alpha (A_\alpha - BK) \\ &= (A_\alpha - BK_\alpha^*)^T P_\alpha + P_\alpha (A_\alpha - BK_\alpha^*) + \Delta K^T P_\alpha + P_\alpha \Delta K \\ &= -Q - K_\alpha^{*T} R K_\alpha^* + \Delta K^T P_\alpha + P_\alpha \Delta K. \end{aligned} \quad (21)$$

From [15, Corollary 2], K_α^* is continuous on α , which implies that P_α is continuous on α . As a result, there exists λ_P such that

$$\lambda_{\max}(P_\alpha) \leq \lambda_P, \quad \alpha \in [0, \alpha_0].$$

Equation (21) means that $A_\alpha - BK$ is stable as long as

$$\lambda_{\max}(\Delta K^T P_\alpha + P_\alpha \Delta K) < \lambda_{\min}(Q),$$

which holds if

$$\|\Delta K\| < \frac{\lambda_{\min}(Q)}{2\lambda_P} \triangleq \delta_1, \quad (22)$$

here $\|\cdot\|$ is the induced 2-norm. ■

Proof of Theorem 1: The updating mechanism of α_{k+1} in Algorithm 2 implies that α_k is nonincreasing in k . Together with $\alpha \geq 0$, we have $\lim_{k \rightarrow \infty} \alpha_k = \alpha^*$ for some $\alpha^* \geq 0$. Note that when $\alpha_k \geq \eta$, α_{k+1} is always obtained by implementing Algorithm 2. It can be observed that the outputs of Algorithm 2 always ensure stability of $A_k - \alpha_k I_n$. However, if $A_{k+1} - (\alpha_k - \eta)I_n$ is not stable, α_{k+1} remains to be α_k while K_{k+1} keeps updating.

We now prove that η can be chosen such that α_k converges to $\alpha^* < \eta$ as $k \rightarrow \infty$. It suffices to show that for any step k , if $\alpha_k \geq \eta$, then there always exists $k' > k$ such that $A - (\alpha_k - \eta)I_n - BK_{k'}$ is stable, and based on $\alpha_k - \eta$ and $K_{k'}$, the updated $P_{k'}$ satisfies $\|P_{k'} - P_{k'-1}\| < \sigma$.

Let $\alpha' = \alpha_k - \eta$. Recall that K_α^* is continuous on α , so K_α^* is uniformly continuous for $\alpha \in [0, \alpha_0]$. Then there exists $\delta_2 > 0$, such that once $|\alpha - \alpha'| < \delta_2$, there must hold that

$$\|K_\alpha^* - K_{\alpha'}^*\| < \delta_1/2,$$

where δ_1 is specified in Lemma 3.

As shown in [7], under Assumption 2, when α_k remains unchanged, and K_k is updated by solving (9), K_k converges to $K_{\alpha_k}^*$ as $k \rightarrow \infty$. That is, there exists $k' > k$ such that

$$\|K_{k'} - K_{\alpha_k}^*\| < \delta_1/2.$$

When $\eta < \delta_2$, we have

$$\|K_{k'} - K_{\alpha'}^*\| \leq \|K_{k'} - K_{\alpha_k}^*\| + \|K_{\alpha_k}^* - K_{\alpha'}^*\| < \delta_1,$$

By Lemma 3, $A - \alpha' I_n - BK_{k'}$ is stable.

On the other hand, note that the solution P_k to (6) is continuous on α_{k+1} and K_k , thus is uniformly continuous for $\alpha \in [0, \alpha_0]$ and $K \in \{K \in \mathbb{R}^{m \times n} : \|K - K_\alpha^*\| < \delta_1\}$. As a result, there is $\delta_3 > 0$, such that if $|\alpha_{k+1} - \alpha_k| < \delta_3$, then $\|P_k - P_{k-1}\| < \sigma$.

Consequently, once we choose $\eta < \min\{\delta_2, \delta_3\}$ and $S_0 \in \mathbb{N}_+$ such that $\alpha_0 = \eta S_0$ renders $A - \alpha_0 I_n$ stable, α_k will converge to $\alpha^* < \eta$ at some step $k'' > 0$, which implies that $\alpha^* = 0$. Continue running Step 3 of Algorithm 1, K_k converges to the optimal control gain K^* as $k \rightarrow \infty$. Moreover, using any integer $S > S_0$ still works because $A - S\eta I_n$ is stable if $A - S_0\eta I_n$ is stable.

REFERENCES

- [1] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [2] N. C. Luong *et al.*, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3133–3174, 4th Quart., 2019.
- [3] T. Sadamoto, A. Chakraborty, and J.-I. Imura, "Fast online reinforcement learning control using state-space dimensionality reduction," *IEEE Trans. Control Netw. Syst.*, vol. 8, no. 1, pp. 342–353, Mar. 2021.
- [4] S. L. Barton and D. Asher, "Reinforcement learning framework for collaborative agents interacting with soldiers in dynamic military contexts," in *Proc. Next Gener. Anal. VI*, vol. 10653, 2018, p. 1065303.
- [5] B. Kiumarsi, K. G. Vamvoudakis, H. Modares, and F. L. Lewis, "Optimal and autonomous control using reinforcement learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 6, pp. 2042–2062, Jun. 2018.
- [6] Z. P. Jiang, T. Bian, and W. Gao, "Learning-based control: A tutorial and some recent results," *Found. Trends Syst. Control*, vol. 8, no. 3, pp. 176–284, 2020.
- [7] Y. Jiang and Z. P. Jiang, "Computational adaptive optimal control for continuous-time linear systems with completely unknown dynamics," *Automatica*, vol. 48, no. 10, pp. 2699–2704, 2012.
- [8] S. J. Bradtke, B. E. Ydstie, and A. G. Barto, "Adaptive linear quadratic control using policy iteration," in *Proc. Amer. Control Conf. (ACC)*, Jun. 1994, pp. 3475–3479.
- [9] A. Lamperski, "Computing stabilizing linear controllers via policy iteration," in *Proc. 59th IEEE Conf. Decis. Control (CDC)*, 2020, pp. 1902–1907.
- [10] M. Fazel, R. Ge, S. M. Kakade, and M. Mesbahi, "Global convergence of policy gradient methods for the linear quadratic regulator," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1466–1475.
- [11] G. Jing, H. Bai, J. George, and A. Chakraborty, "Model-free optimal control of linear multi-agent systems via decomposition and hierarchical approximation," *IEEE Trans. Control Netw. Syst.*, to be published.
- [12] S. Alemzadeh and M. Mesbahi, "Distributed Q-learning for dynamically decoupled systems," in *Proc. Amer. Control Conf. (ACC)*, 2019, pp. 772–777.
- [13] Y. Li, Y. Tang, R. Zhang, and N. Li, "Distributed reinforcement learning for decentralized linear quadratic control: A derivative-free policy optimization approach," 2019. [Online]. Available: arXiv:1912.09135.
- [14] C. De Persis and P. Tesi, "On persistency of excitation and formulas for data-driven control," in *Proc. IEEE 58th Conf. Decis. Control (CDC)*, 2019, pp. 873–878.
- [15] H. Feng and J. Lavaei, "Escaping locally optimal decentralized control policies via damping," in *Proc. Amer. Control Conf. (ACC)*, 2020, pp. 50–57.
- [16] D. Kleinman, "On an iterative technique for Riccati equation computations," *IEEE Trans. Autom. Control*, vol. AC-13, no. 1, pp. 114–115, Feb. 1968.
- [17] D. Boskos and D. V. Dimarogonas, "Finite horizon discrete models for multi-agent control systems with coupled dynamics," *Automatica*, vol. 115, May 2020, Art. no. 108838.
- [18] I. Shames, A. M. H. Teixeira, H. Sandberg, and K. H. Johansson, "Distributed fault detection for interconnected second-order systems," *Automatica*, vol. 47, no. 12, pp. 2757–2764, 2011.